

CS 639 Final Project: LLM Evaluation of Hackathon Submissions

Team Members: Ethan Feiges, George Sukhotin, Mari Garey, Oliver Ohrt, Manas Joshi, Andrew Lou, Ben Friedl, James Ding

1. Introduction

Hackathons are competitions where individuals compete to create the best project matching some criteria within a short period of time. Evaluating which projects are the best requires a large amount of human effort, which can be difficult to find. We seek to evaluate if LLMs are capable of judging hackathons in a way that agrees with human judging results. We hope to improve the consistency of hackathon judging as well, eliminating human inconsistencies in judging.

2. Hypothesis and Research Questions

Our hypothesis can be broken down in multiple components:

- 1) Can the model effectively compare two projects and determine which one is better?
- 2) Can the model fine-tune based off of the frontier model's responses, determining strong/weak attributes of a project?
- 3) Can the model develop a universal hackathon ranking system?

3. Related Work

Our project connects multiple different components in machine learning.

3.1 LLM-as-Judge

The core concept of LLM-as-Judge is central to our project, leveraging Large Language Models (LLMs) to automatically evaluate the outputs of hackathon submissions. Research has shown a high correlation with human preferences in tasks like chatbot quality assessment. This is crucial for hackathon judging since human-evaluation has potentially detrimental bottlenecks which are addressed by LLM-as-Judge. By framing the judging process as a pairwise comparison and structured critique task, we are directly applying the LLM-as-Judge to eliminate human

inconsistencies and determine project rankings. LLM-as-judge does tend to have position bias, which we detect during our experiment.

3.2 Qwen3-4b model

The Qwen3-4B model, which serves as the base for our fine-tuning methodology, is detailed in the *Qwen3 Technical Report*. Qwen3 includes models of both dense and Mixture of Expert architectures, the latter splits work across several smaller and specialized sub network “experts” to improve latency and scalability, as more experts can be added than making the entire model wider or deeper. Additionally, the Qwen3 series adds thinking mode to enable dynamic switching between chat-optimized models and dedicated reasoning models. Qwen3 models demonstrate exceptional performance across multiple benchmarks such as in code generation and agent tasks.

The choice of the Qwen3-4B variant is relevant to our project for it is a smaller, more efficient model emphasizing STEM and logical reasoning in its training. This makes it an ideal candidate for fine-tuning in a hackathon judging context under limited compute resources. Additionally, the General RL stage targets instruction following and preference alignment, which is exceptionally important for judging hackathons.

3.3 Supervised Fine-Tuning and LoRA

Supervised Fine-Tuning (SFT) is a crucial initial step in alignment, where a pre-trained model is trained on a high-quality dataset of prompt-response examples that demonstrate the desired behavior. This process fine-tunes the model's weights to follow instructions and generate responses in a target format. For our project, SFT is essential for teaching the Qwen3-4B model the necessary structure and tone for hackathon judging using the LLM-generated structured critique and numerical scores, establishing the fundamental instruction-following capability required before further preference alignment.

We use LoRA to ensure we can adapt a model without retraining all of its parameters. LoRA accomplishes this by injecting two smaller trainable matrices with rank r into the Transformer layers. This reduces the number of trainable parameters by 10,000 times and the GPU memory requirement by 3 times and has proven to be on-par with fine-tuning much larger models like GPT-2 and GPT-3. However, its advantage lies in a reduced memory footprint and zero inference latency since LoRA can be merged with frozen weights for deployment. LoRA is compatible with the Qwen3 model since it reduces the memory footprint and maintains base reasoning abilities while learning on the specific hackathon judging task.

3.4 Bradley-Terry comparisons

Bradley-Terry comparisons provide a standard framework for learning rankings from pairwise preferences. Rather than giving an absolute score to each project, the model observes comparisons between two different submissions, and is able to learn a latent quality score for each project. There's a probability that project i is preferred over project j and is modelled as $P(i > j) = \sigma(s_i - s_j)$. s_i and s_j are the project's latent quality scores. This becomes especially useful for hackathon judging because the task is naturally comparative: a human judge decides it based on creativity, technical implementation, usefulness, and presentation. Our project has scraped Devpost results converted into pairwise comparisons so the model can learn relative judging behavior and produce rankings from those comparisons, directly supporting our goal of fine tuning a smaller model to compare hackathon submissions and approximate judging models.

4. Methodology

Our approach is separated in three separate steps:

4.1 Hackerrank Scraping

One of the most crucial steps in our project workflow was gathering training data for model fine-tuning. 'Data-Centric AI in the Age of Large Language Models' by Xu et al (2024) assigns utmost importance to good quality data in LLM development, arguing that data is central to model capability and deserves more attention than it often gets. To ensure that our data reflects the latest real-world trends with regards to hackathon rankings, we have chosen to obtain most of it from real publicly-available hackathon records on Devpost. Devpost is the leading online platform for participating in and facilitating hackathons. It was identified as the best source for raw data as it stores submissions for most large-scale hackathons in the nation, which would expose our models to a variety of hackathon judging decisions. The format in which the submission data is stored is uniform across all hackathons regardless of the host or type, facilitating a consistent and scalable raw data collection pipeline.

Gathering the raw data consists of the following main parts: parsing the hackathon, parsing the projects, constructing a structured record for each project, and saving each record to a JSON file. The main tools that we used for our hackathon scraper include BeautifulSoup at the web page parsing and data extraction layer, playwright at the browser automation layer, and JSON at the storage and serialization layer. The scraper takes the subdomain of the hackathon on devpost, navigates to the submissions section of the page, and retrieves the relevant project information including but not limited to the 'title', 'description', 'built with', and 'results'. The retrieved data is stored in a JSON file to ensure smooth access and maximum scalability.

After the raw data was obtained and stored in an accessible format, the data was cleaned of errors and corrupted data points which result from incidental page timeouts during scraping. For each hackathon file, the remaining valid data points were used to create pairwise comparisons to ensure compatibility with the Bradley-Terry model and downstream model fine-tuning. In an effort to add a form of regularization to the training data, the number of pairs for each hackathon was randomized and bounded to prevent overrepresentation. The variance of winner/non-winner positions in each pair sought to mitigate the development of shortcuts, emphasizing extraction and examination of meaningful project contents instead.

This pipeline was run on roughly 10 different hackathons, most of which took place between 2024 and 2026, ensuring that the dataset reflects most recent trends in hackathon judging. This gave us upwards of 2,000 submissions across all hackathons to work with, which resulted in around 10,000 pairwise comparisons for model fine-tuning. Our work with the data for this project has shown that the internet, a customary source for LLM training, remains a strong economical source for scalable, high-quality data that reflects the latest real-world patterns. We have also seen that a well-designed pipeline can effectively transform publicly available web-formatted records into structured training data suitable for model fine-tuning.

4.2 LLM evaluation

The LLM evaluation step was critical for generating the high-quality, structured critique dataset required for Supervised Fine-Tuning (SFT). This process used a frontier model, specifically the Qwen3.5-72B, as the "teacher" to evaluate the raw pairwise comparisons gathered during the scraping phase. The frontier model was tasked with two main objectives: performing a pairwise comparison to determine the superior project, and generating a detailed structured critique with numerical scores for each submission. Framing the judging process as a pairwise comparison and structured critique task is a direct application of the LLM-as-Judge paradigm, which has been shown to produce more reliable and calibrated quality signals than absolute scoring alone, since relative comparisons are cognitively easier to make consistently even for large models.

Statistical Methods

To evaluate judge quality, we use two distinct ground-truth signals depending on what is available for each hackathon. For all eight hackathons, DevPost winner labels provide a binary reference: a project is either a winner or not. For madhacks-fall-2025 only, human judges submitted ranked-choice ballots, enabling a full reference ranking via Plackett-Luce. Our metrics are designed to exploit whichever signal is available.

Bradley-Terry ranking via ILSR. Each judge's pairwise verdicts are converted into a global project ranking by fitting a Bradley-Terry model using Iterative Luce Spectral Ranking (choix.ilsr_pairwise, $\alpha=0.01$). This aggregates potentially inconsistent pairwise comparisons into a single latent strength score per project, which is more robust than simple win-rate ranking because it accounts for the transitivity structure of the comparison graph and handles cycles.

The regularization parameter α prevents degeneracy in sparse or disconnected pair graphs by adding a small virtual mass to every edge. Ties are encoded as one win in each direction.

Human Plackett-Luce reference (madhacks-fall-2025 only). For this hackathon, human judges submitted ranked-choice ballots. A Plackett-Luce model is fit to those ballots via maximum a posteriori estimation using PyMC, yielding a continuous latent score per project that serves as the reference against which judges' BT scores are compared. This reference does not exist for the other seven hackathons, where only binary winner labels are available.

Recall@K: the fraction of DevPost winners that appear in a judge's BT top-K. We report it at multiple values of K to distinguish between judges that identify a small set of clearly excellent projects versus those that only agree with the human signal at larger, easier cutoffs. Bootstrap confidence intervals are reported to quantify uncertainty about which judgment pairs happened to be observed.

Rank correlation (Spearman's ρ and Kendall's τ). For madhacks-fall-2025, where a continuous human PL reference exists, we compute Spearman's ρ and Kendall's τ between each judge's BT log-skill scores and the human PL scores. Spearman's ρ measures monotonic agreement across the full ranking and is sensitive to large positional errors; Kendall's τ penalizes each pairwise inversion independently and is interpretable as the probability that a randomly chosen pair is ordered consistently between a judge and the human reference. These metrics are also reported between pairs of judges across all hackathons as a measure of inter-judge agreement.

Distillation arc bootstrap. For madhacks-fall-2025, we directly test whether fine-tuning improved over the base model or closed the gap to the teacher by computing paired Spearman ρ differences across $B=1000$ bootstrap iterations. Each iteration resamples judgment pair IDs with replacement, refits BT for all judges on the resampled pairs, and computes ρ against the fixed human PL reference. We report the mean $\Delta\rho$ with a 95% percentile bootstrap interval and $P(\Delta>0)$ (the fraction of iterations in which the fine-tuned model outperforms the comparison). The bootstrap is used because the distribution of ρ differences across a small sample does not satisfy normality assumptions.

Top-K and bottom-K intersection. We report raw overlap between each judge's top-K and bottom-K lists and the human PL reference at multiple cutoffs. This makes the practical implications of the correlation metrics concrete: translating abstract ρ values into the actual projects a judge would select or eliminate. This also helps diagnose whether judges fail specifically at the top, the bottom, or uniformly across the ranking.

4.3 Fine-tuning

The objective for this is to determine whether we can fine tune a smaller model, like the Qwen3.5-4B, to evaluate hackathons equivalently to a frontier model. We use an 80/20 train/test

split on a dataset of frontier LLM judging responses and pairwise project comparisons, applying consistency filtering and pair-based splitting. Consistency filtering removes any pairs where the frontier model's verdict changed depending on which project was presented first, retaining only judgments that reflect genuine quality differences rather than presentation-order artifacts. Pair-based splitting ensures that both position-swapped variants of a given comparison (A vs. B and B vs. A) always land in the same split, preventing data leakage that would artificially inflate test agreement.

To train, we use LoRA to fine-tune the weights using typical parameters: batch size of 4, learning rate of $3e-4$, and 3 epochs. However, we set the rank of LoRA to 32 and set the gradient accumulation steps to 8, which will create an effective batch size of 32 and help the model capture more nuanced weights for the complex task of judging. We also set the `max_seq_length` to 12288 to support long training examples from the dataset, filtering out those that exceed this limit. The large sequence lengths are counteracted using Liger kernels and flash attention to optimize for memory. We utilize the PEFT library by Hugging Face to wrap the standard transformer model around our LoRA configurations.

Before training, we preprocess each example by applying the chat template with thinking enabled and generation prompt disabled, converting the structured message list into a single tokenized string that includes the frontier model's reasoning trace. A Completion-Only Data Collator then masks all tokens prior to the assistant turn with a loss weight of -100, ensuring the model's training signal comes exclusively from learning to reproduce the assistant's output rather than the prompt. Training follows a cosine learning rate schedule with a 3% warmup, which prevents large destabilizing gradient steps in the early phase when the LoRA weights are near zero, then smoothly decays the learning rate as the adapter converges.

(Due to limited time and compute, we ended up switching from Qwen3.5-4B -> Qwen3-4B instruct, 3 epochs -> 2 epochs, $r=32$ -> $r=16$. This reduced train time estimates from ~100 hours to ~20 hours)

5. [Results](#)

5.1 Data

To assess the performance of our fine-tuned judge, [devpost-hacks-qwen3-4b-judge](#), we compared it against four other models: our baseline (Qwen3-4B-Instruct), a frontier comparison (GPT-OSS-20B), and two additional variants (Qwen3.5-4B and Qwen3.5-27B). Evaluations were conducted using a held-out test set comprising 2,430 project pairs over 8 hackathons from a

fixed seed split. All metrics below are computed on this held-out set with 1,000-iteration bootstrap confidence intervals unless noted otherwise.

Latency and Tokens

The fine-tuned model used far more tokens than the base when reasoning, with a median length of 2,953 compared to 1,192 for the base. The fine-tuned model also had significantly higher truncation rates (16.36% >8192 tokens) than every other model, and had a significantly higher median latency than every model except for its teacher, Qwen3.5-27B, which had even higher latency.

Latency + token stats per judge

model	n	median compl tok	p95 compl tok	median lat s	p95 lat s	truncated %
Qwen/Qwen3-4B-Instruct-2507	18,608	1,192	1,446	36.3	53.0	0.00%
Qwen/Qwen3.5-27B	31,522	2,763	5,270	190.9	355.1	0.56%
Qwen/Qwen3.5-4B	31,522	3,434	7,565	60.7	133.2	3.61%
openai/gpt-oss-20b	25,539	769	1,556	53.7	173.0	0.00%
twangodev/devpost-hacks-qwen3-4b-judge	31,544	2,953	8,192	124.6	349.6	16.36%

Verdict Distribution and Consistency

Judges differed substantially in their decision patterns. The base Qwen3-4B-Instruct-2507 showed a strong positional bias, picking "B" 58.7% of the time, suggesting sensitivity to presentation order. The fine-tuned model showed a preference for position "A" (+9%), likely picking up on the position bias of its teacher, which also showed a strong preference for "A" (+14%). Our fine-tuned model introduced a surprising 5.5% tie rate, the highest of any judge, reflecting a learned reluctance to force a winner despite neither the base nor the teacher showing this behavior. Position-consistency tests (presenting the same pair in both AB and BA orderings) revealed that Qwen3.5-4B and gpt-oss-20b were most consistent (87.4% and 86.3%, respectively), while the base 2507 model was the least consistent at 74.9%. Our fine-tune achieved 84.8%, an improvement over its base but trailing the stronger open-weights peers. These results reveal an unexpected point of failure in our methodology, as the smaller Qwen3.5-4B showed a +7% higher consistency over our teacher, Qwen3.5-27B, putting into question whether we selected the best model to be the teacher.

Verdict distribution per judge

model	A	B	tie	invalid	A%	B%	tie%	inv%
Qwen/Qwen3-4B-Instruct-2507	7,679	10,929	0	0	41.3%	58.7%	0.0%	0.0%
Qwen/Qwen3.5-27B	17,632	13,235	0	655	55.9%	42.0%	0.0%	2.1%
Qwen/Qwen3.5-4B	15,089	15,302	3	1,128	47.9%	48.5%	0.0%	3.6%
openai/gpt-oss-20b	12,460	12,880	47	152	48.8%	50.4%	0.2%	0.6%
twangodev/devpost-hacks-qwen3-4b-judge	15,882	12,943	1,733	986	50.3%	41.0%	5.5%	3.1%

Position consistency (A/B vs B/A picks same project)

model	pairs	agree	flip	inv-1	inv-2	consistency	flip rate
Qwen/Qwen3.5-4B	15,761	12,817	1,851	1,056	37	87.4%	12.6%
openai/gpt-oss-20b	12,764	10,848	1,718	198	0	86.3%	13.7%
twangodev/devpost-hacks-qwen3-4b-judge	15,772	11,171	2,001	2,579	21	84.8%	15.2%
Qwen/Qwen3.5-27B	15,761	12,151	2,965	635	10	80.4%	19.6%
Qwen/Qwen3-4B-Instruct-2507	9,304	6,967	2,337	0	0	74.9%	25.1%

Top-1 Agreement with Devpost Winners

Top-1 agreement proved difficult across the board. Aggregated over 8 hackathons, no model exceeded 25% top-1 hit rate. gpt-oss-20b, Qwen3.5-27B, and Qwen3.5-4B each correctly identified a winner at rank 1 in 2 out of 8 hackathons (point estimate), while the fine-tuned model never did. The bootstrap hit rates (a more reliable estimate of how often rank-1 is a winner across resampled pair sets) show **significant hackathon-level and model-level variance**: on PennApps XXV, gpt-oss-20b achieved a bootstrap hit rate of 81%, with the worst performing model (our fine-tuned) having a hit rate of 38%. Meanwhile, on Madhacks Fall 2025, the hit rates fell between 11% and 6%

Spearman ρ against Human Plackett-Luce Rankings (MadHacks)

On MadHacks Fall 2025, the hackathon where we had access to human Plackett-Luce rankings, all models produced moderate positive correlations with human preferences: gpt-oss-20b led at $\rho = +0.335$ [0.291, 0.383], followed by the base 2507 at $\rho = +0.334$ [0.289, 0.379], Qwen3.5-4B at +0.328, our fine-tune at +0.307, and Qwen3.5-27B at +0.294. All confidence intervals are comfortably above zero, indicating some level of agreement with human ranking signals above chance across all judges. These results once again raise a concern as to whether we picked the right teacher model as the teacher had the worst performance out of all models evaluated and our fine-tuned model underperformed our base model, albeit the difference is not statistically significant.

model	Spearman ρ [95% CI]
openai/gpt-oss-20b	+0.335 [+0.291, +0.383]
Qwen/Qwen3-4B-Instruct-2507	+0.334 [+0.289, +0.379]
Qwen/Qwen3.5-4B	+0.328 [+0.281, +0.372]
twangodev/devpost-hacks-qwen3-4b-judge	+0.307 [+0.252, +0.359]
Qwen/Qwen3.5-27B	+0.294 [+0.246, +0.337]

Distillation Arc

A paired bootstrap comparison between the fine-tune, its base, and its teacher (Qwen3.5-27B) showed: the fine-tune underperformed the base by $\Delta p = -0.027$ $[-0.078, +0.021]$ with only a 12.9% probability that the fine-tune exceeds the base. Against the teacher, the fine-tune trailed by $\Delta p = -0.014$ with a 71.5% probability of improvement. While not definitive, this reinforces our concerns above. Pairwise Spearman ρ between all judge pairs (medians across shared hackathons) was uniformly high, ranging from $+0.784$ to $+0.911$. The closest pairing was Qwen3.5-27B and our fine-tuned model at $\rho = +0.911$, $\tau = +0.765$, confirming that distillation changed the fine-tuned behavior significantly as the pairing between our base and the teacher had $\rho = +0.784$, $\tau = +0.623$, the lowest agreement amongst all pairs.

comparison	Δp [95% CI]	$P(\Delta > 0)$
ft minus base	-0.0271 $[-0.078, +0.021]$	12.9%
ft minus teacher	$+0.0135$ $[-0.035, +0.063]$	71.5%

judge A	judge B	N hacks	median ρ	median τ
Qwen/Qwen3-4B-Instruct-2507	Qwen/Qwen3.5-27B	7	$+0.784$	$+0.623$
Qwen/Qwen3-4B-Instruct-2507	Qwen/Qwen3.5-4B	7	$+0.807$	$+0.631$
Qwen/Qwen3-4B-Instruct-2507	openai/gpt-oss-20b	7	$+0.816$	$+0.641$
Qwen/Qwen3-4B-Instruct-2507	twangodev/devpost-hacks-qwe...	7	$+0.810$	$+0.642$
Qwen/Qwen3.5-27B	Qwen/Qwen3.5-4B	8	$+0.861$	$+0.711$
Qwen/Qwen3.5-27B	openai/gpt-oss-20b	8	$+0.795$	$+0.638$
Qwen/Qwen3.5-27B	twangodev/devpost-hacks-qwe...	8	$+0.911$	$+0.765$
Qwen/Qwen3.5-4B	openai/gpt-oss-20b	8	$+0.828$	$+0.650$
Qwen/Qwen3.5-4B	twangodev/devpost-hacks-qwe...	8	$+0.845$	$+0.677$
openai/gpt-oss-20b	twangodev/devpost-hacks-qwe...	8	$+0.811$	$+0.629$

MadHacks Top/Bottom-K Overlap

The top-K and bottom-K intersection with human PL rankings was not very impressive across all judges. At $K=5$, every model scored 0/5 overlap with the human top-5, and 1/5 overlap with the human bottom-5. At $K=50$, all models converged to 54–58% overlap, but this number is not very impressive given the dataset size. With only 113 total projects, selecting the top or bottom 50 covers nearly half the field, making a random baseline of $50/113 \approx 44\%$ the appropriate

comparison. The observed 54–58% overlap represents only a modest improvement over random selection.

PL top-K n judge top-K (madhacks-fall-2025)

model	K=5	K=10	K=20	K=50
2507 (base)	0/5 (0%)	1/10 (10%)	6/20 (30%)	28/50 (56%)
gpt-oss-20b	0/5 (0%)	0/10 (0%)	5/20 (25%)	29/50 (58%)
FT (4b-judge)	0/5 (0%)	2/10 (20%)	8/20 (40%)	27/50 (54%)
Qwen3.5-4B	0/5 (0%)	0/10 (0%)	5/20 (25%)	27/50 (54%)
Qwen3.5-27B	0/5 (0%)	1/10 (10%)	5/20 (25%)	27/50 (54%)

PL bottom-K n judge bottom-K (madhacks-fall-2025)

model	K=5	K=10	K=20	K=50
2507 (base)	1/5 (20%)	3/10 (30%)	5/20 (25%)	28/50 (56%)
gpt-oss-20b	1/5 (20%)	2/10 (20%)	6/20 (30%)	29/50 (58%)
FT (4b-judge)	1/5 (20%)	1/10 (10%)	6/20 (30%)	28/50 (56%)
Qwen3.5-4B	1/5 (20%)	1/10 (10%)	8/20 (40%)	28/50 (56%)
Qwen3.5-27B	1/5 (20%)	2/10 (20%)	5/20 (25%)	27/50 (54%)

5.2 Interpretation and Relationship to Research Questions

Can the model effectively compare two projects and determine which is better?

All five judges produced rankings weakly correlated with human preferences ($\rho \approx 0.29$ – 0.34 on MadHacks), demonstrating that pairwise comparison captures a weak signal above chance. However, position-consistency rates of 75–87% reveal that the same model will reverse its verdict on 13–25% of pairs merely by swapping presentation order. That’s not to say that humans don’t exhibit the same biases, but we can expect a computer to be able to be more impartial when it comes to these things.

Can the model fine-tune based on frontier model responses, learning strong/weak attributes of a project?

The distillation results reveal a critical flaw in the project's core assumption: the teacher model (Qwen3.5-27B) was itself the weakest judge by human alignment, achieving the lowest Spearman ρ at +0.294 — below the base model's +0.334, below gpt-oss-20b's +0.335, and below even Qwen3.5-4B's +0.328. This was not anticipated during planning, where teacher size was assumed to correlate with judgment quality. In retrospect, a larger reasoning model is not necessarily better calibrated to human hackathon preferences; it may simply be more confident in its own internally consistent but human-misaligned criteria. The base model (Qwen3-4B instruct) in this case was actually already a fine-tuned version of Qwen3-4B for instruction-following and complex reasoning that we decided to use because we found we wouldn't have sufficient time to fine tune Qwen3.5-4B, which was our original time. In a way, our results suggest that the instruct model (base which was already fine tuned) was able to have similar performance in hackathon-judging relative to our best model (gpt-oss-20b) whilst having less parameters and fewer latency. It's just that when we tried to fine tune it even further specifically on hackathon-judging, we chose a teacher that it already outperformed.

Can the model develop a universal hackathon ranking system?

The cross-hackathon results suggest the answer is currently no. Hackathon-level variance in bootstrap top-1 hit rate is enormous ranging from 4% to 81% across (model $\times$ judge) combinations. The differences between models wash out when averaged across hackathons, meaning no judge has demonstrated a generalizable ranking advantage.

The high inter-judge correlations (all $\rho > 0.78$) suggest the models have converged on a shared internal notion of project quality that is consistent across judges but not reliably aligned to human ground truth. The MadHacks top/bottom-K overlap results reinforce this: at K=5, every model scores 0/5 overlap with human top-5 winners. At K=10, the best result is 2/10 (20%), a modest lift above the random baseline of ~9%. At K=50, apparent overlap rates of 54–58% are barely above the random baseline of 44% given only 113 total projects, and should not be interpreted as evidence of meaningful discrimination.

Taken together, these findings suggest that a universal ranking system built on any of these judges would be unreliable for replicating human placement decisions.

6. Conclusions and Future Work

This project investigated whether LLMs can judge hackathons in agreement with human evaluators, across three sub-questions: whether a model can use pairwise comparison

effectively to compare two projects, whether distillation from a frontier model can produce a reliable small judge, and whether a universal ranking system is achievable. Our approach did not yield results that indicate a strong “yes” to any of these questions. Position bias is not uncommon across pairs, and Spearman ρ values and top-K overlap suggests weak correlation with human ranking. Distillation worked, but we chose the wrong model to distill from, making our fine-tuned model underperform the base when compared on the basis of human-rankings.

For future work, two directions stand out as most promising. First, **teacher selection** is the most immediate fix. Rather than assuming the largest available model produces the best judgments, future work should empirically validate teacher alignment against human ground truth before distillation. Second, **richer input modalities** could substantially close the gap between LLM and human judges. Human evaluators at hackathons watch live demos, ask questions, and assess presentation quality. Incorporating video demos, slides, or structured interviewer notes as additional context could give judges access to the same signals humans rely on.

7. Citations

LLM-as-Judge (3.1): Zheng, Lianmin, et al. “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena.” *arXiv.Org*, 2023, <https://arxiv.org/abs/2306.05685>.

Qwen (3.2): Qwen Team. “Qwen3 Technical Report.” *arXiv.Org*, 2025, <https://arxiv.org/abs/2505.09388>.

LoRA (3.3): Hu, Edward J., et al. “LoRA: Low-Rank Adaptation of Large Language Models.” *arXiv.Org*, 2021, <https://arxiv.org/abs/2106.09685>.

SFT (3.3): Ouyang, Long, et al. “Training Language Models to Follow Instructions with Human Feedback.” *arXiv.Org*, 2022, <https://arxiv.org/abs/2203.02155>.

Bradley-Terry comparisons (3.4): Bradley, Ralph Allan, and Milton E. Terry. “Rank Analysis of Incomplete Block Designs: I. The Method of Paired Comparisons.” *Biometrika*, vol. 39, no. 3/4, 1952, pp. 324–345. <https://doi.org/10.2307/2334029>

Scraping (4.1): Xu, Xinyi, et al. “Data-Centric AI in the Age of Large Language Models.” *arXiv.Org*, 2024, <https://arxiv.org/abs/2406.14473>.